

Reducing Login Alert Fatigue in Wazuh Using Safe Rule Overrides

Project Goal

The goal of this project is to **reduce alert fatigue caused by frequent successful login alerts**, while **preserving security visibility and detection of real attacks**, using **safe Wazuh rule overrides**.

This project focuses on **severity tuning**, not alert suppression.

Problem Statement

In a SOC environment, the Wazuh manager was generating **multiple high-severity alerts** for:

- Successful login during non-business hours
- Successful login during weekends

These alerts appeared:

- Repeatedly in **Wazuh**
- Automatically in **TheHive** as alerts
- With **high severity (level 9)**

Impact

- SOC analysts were flooded with **non-actionable alerts**
- Real threats risk being ignored due to alert fatigue
- Time was wasted investigating legitimate administrative logins

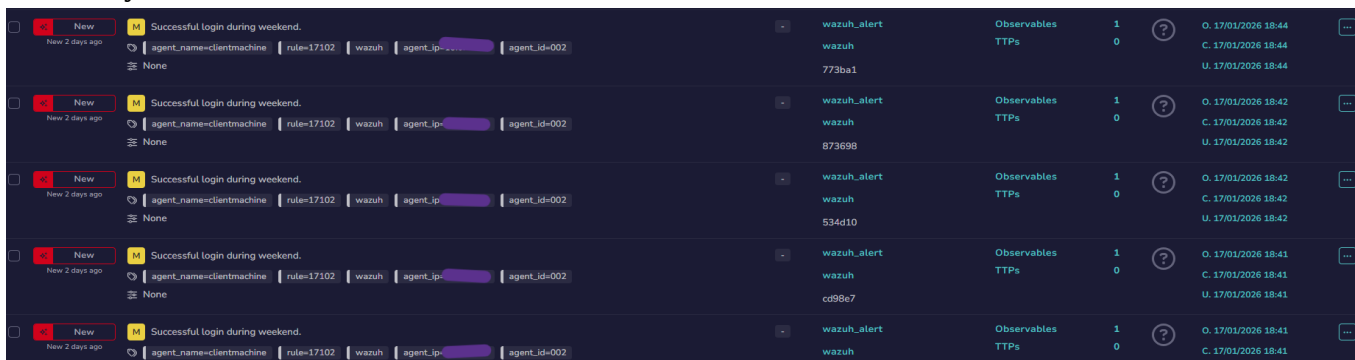
Initial Observation (Before Tuning)

In Wazuh

- Rule ID: 17101 and 17102
- Rule Level: **9 (High)**
- Triggered for:
 - Normal admin SSH logins
 - Maintenance activities
 - Automated jobs

In TheHive

- Multiple alerts created for the same type of activity
- Analysts forced to review non-malicious events



		Successful login during weekend.	 agent_name=clientmachine rule=17102 wazuh agent_ip= agent_id=002		wazuh_alert	Observables	1		O. 17/01/2026 18:44	
			 agent_name=clientmachine rule=17102 wazuh agent_ip= agent_id=002		wazuh	TTPs	0		C. 17/01/2026 18:44	
			 agent_name=clientmachine rule=17102 wazuh agent_ip= agent_id=002		773ba1				U. 17/01/2026 18:44	
		Successful login during weekend.	 agent_name=clientmachine rule=17102 wazuh agent_ip= agent_id=002		wazuh_alert	Observables	1		O. 17/01/2026 18:42	
			 agent_name=clientmachine rule=17102 wazuh agent_ip= agent_id=002		wazuh	TTPs	0		C. 17/01/2026 18:42	
			 agent_name=clientmachine rule=17102 wazuh agent_ip= agent_id=002		873698				U. 17/01/2026 18:42	
		Successful login during weekend.	 agent_name=clientmachine rule=17102 wazuh agent_ip= agent_id=002		wazuh_alert	Observables	1		O. 17/01/2026 18:42	
			 agent_name=clientmachine rule=17102 wazuh agent_ip= agent_id=002		wazuh	TTPs	0		C. 17/01/2026 18:42	
			 agent_name=clientmachine rule=17102 wazuh agent_ip= agent_id=002		534d10				U. 17/01/2026 18:42	
		Successful login during weekend.	 agent_name=clientmachine rule=17102 wazuh agent_ip= agent_id=002		wazuh_alert	Observables	1		O. 17/01/2026 18:41	
			 agent_name=clientmachine rule=17102 wazuh agent_ip= agent_id=002		wazuh	TTPs	0		C. 17/01/2026 18:41	
			 agent_name=clientmachine rule=17102 wazuh agent_ip= agent_id=002		cd98e7				U. 17/01/2026 18:41	

```
> Jan 28, 2026 @ 09:44:19.848 predecoder.hostname: ip- predecoder.program.name: su predecoder.timestamp: Jan 28 04:14:18 input.type: log agent.ip: agent.name: clientmachine agent.id: 002 manager.name: su
data.srcuser: ubuntu data.uid: 0 data.dstuser: root rule.mail: false rule.level: 9 rule.pci_dss: 10.2.5, 10.6.1 rule.hipaa: 164.312.b rule.tsc: CC5.8, CC7.2, CC7.3 rule.description: Successful login during non
-business hours rule.groups: policy.violation, login.time rule.nist_800_53: AU.14, AC.7, AU.6 rule.gdpr: IV.35.7.d, IV.32.2 rule.firetimes: 2 rule.mitre.technique: Valid Accounts rule.mitre.id: T1078
rule.mitre.tactic: Defense Evasion, Persistence, Privilege Escalation, Initial Access rule.id: 17101 rule.gpg13: 7.1, 7.2 location: journald decoder.parent: pam decoder.name: pam id: 1768882459.82884
full_log: Jan 28 04:14:18 ip su[2100]: pam_unix(su:session): session opened for user root(uid=0) by ubuntu(uid=0) timestamp: Jan 28, 2026 @ 09:44:19.848 index: wazuh-alerts-4.x-2026.01.28

> Jan 28, 2026 @ 09:44:19.848 predecoder.hostname: su predecoder.program.name: sudo predecoder.timestamp: Jan 28 04:14:18 input.type: log agent.ip: 1f agent.name: clientmachine agent.id: 002 manager.name: su
data.srcuser: ubuntu data.uid: 1000 data.dstuser: root rule.mail: false rule.level: 9 rule.pci_dss: 10.2.5, 10.6.1 rule.hipaa: 164.312.b rule.tsc: CC5.8, CC7.2, CC7.3 rule.description: Successful login during
non-business hours rule.groups: policy.violation, login.time rule.nist_800_53: AU.14, AC.7, AU.6 rule.gdpr: IV.35.7.d, IV.32.2 rule.firetimes: 1 rule.mitre.technique: Valid Accounts rule.mitre.id: T1078
rule.mitre.tactic: Defense Evasion, Persistence, Privilege Escalation, Initial Access rule.id: 17101 rule.gpg13: 7.1, 7.2 location: journald decoder.parent: pam decoder.name: pam id: 1768882459.82884
full_log: Jan 28 04:14:18 ip sudo[2098]: pam_unix(sudo:session): session opened for user root(uid=0) by ubuntu(uid=1000) timestamp: Jan 28, 2026 @ 09:44:19.848 index: wazuh-alerts-4.x-2026.01.28
```

 SOC Analysis & Decision

Successful logins **by themselves** are **not incidents**.

In real SOC operations:

- A successful login becomes suspicious **only when combined with other behaviors**
- Example:
 - Login + sudo usage
 - Login + account modification
 - Login + persistence activity

 SOC Decision

Reduce the **severity** of successful login alerts instead of disabling them.

 Solution Approach

We implemented **safe rule overrides** using:

- local_rules.xml
- overwrite="yes" attribute
- No modification of core Wazuh rule files

This ensures:

- Upgrade safety
- Configuration transparency
- Reversible tuning

RULE ID	PURPOSE	ACTION
5503	Failed login	Severity tuned
17101	Login during non-business hours	Severity reduced
17102	Login during weekend	Severity reduced

 No rules were deleted or disabled.

🌸 Step By Step Implementation

📄 Rule Override Configuration

```
<group name="linux,ssh,override_demo">

  <!-- Override Rule 5501: Session opened -->
  <rule id="5501" level="2" overwrite="yes">
    <if_sid>5500</if_sid>
    <match>session opened for user</match>
    <description>PAM: Login session opened (custom level 2)</description>
  </rule>

  <!-- Override Rule 5503: Failed login -->
  <rule id="5503" level="3" overwrite="yes">
    <if_sid>5500</if_sid>
    <match>authentication failure</match>
    <description>PAM: User login failed (custom level 3)</description>
  </rule>

  <rule id="17101" level="5" overwrite="yes">
    <if_sid>5501</if_sid>
    <description>
      Successful login during non-business hours (severity reduced)
    </description>
  </rule>

  <rule id="17102" level="5" overwrite="yes">
    <if_sid>5501</if_sid>
    <description>
      Successful login during weekend (severity reduced)
    </description>
  </rule>

</group>
```

🧪 Validation & Testing

All changes were validated using:

✅ Test 1: Failed Login

Input:

pam_unix(sshd:auth): authentication failure

Result:

- Rule ID: 5503
- Level: 3
- Confirms override is active

```
Jan 20 13:10:00 clientmachine sshd[2222]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.50.100

**Phase 1: Completed pre-decoding.
Full event: 'Jan 20 13:10:00 clientmachine sshd[2222]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.50.100'
timestamp: 'Jan 20 13:10:00'
hostname: 'clientmachine'
program_name: 'sshd'

**Phase 2: Completed decoding.
name: 'pam'
euid: '0'
srcip: '192.168.50.100'
tty: 'ssh'
uid: '0'

**Phase 3: Completed filtering (rules).
id: '5503'
level: '3'
description: 'PAM: User login failed (custom level 3)'
groups: '['linux', 'ssh', 'override_demo']'
firedtimes: '2'
mail: 'False'

**Alert to be generated.
```

✅ Test 2: Session Opened (Non-Business Hours)

Input:

pam_unix(sshd:session): session opened for user ubuntu

Result:

- Rule ID: 17101
- Level: **5**
- Severity successfully reduced

```
Jan 20 13:11:00 clientmachine sshd[12345]: pam_unix(sshd:session): session opened for user ubuntu by (uid=0)

**Phase 1: Completed pre-decoding.
  full event: 'Jan 20 13:11:00 clientmachine sshd[12345]: pam_unix(sshd:session): session opened for user ubuntu by (uid=0)'
  timestamp: 'Jan 20 13:11:00'
  hostname: 'clientmachine'
  program_name: 'sshd'

**Phase 2: Completed decoding.
  name: 'pam'
  parent: 'pam'
  dstuser: 'ubuntu'
  uid: '0'

**Phase 3: Completed filtering (rules).
  id: '17101'
  level: '5'
  description: '          Successful login during non-business hours (severity reduced)'
  ,
  groups: '['linux', 'ssh', 'override_demo']'
  firetimes: '1'
  mail: 'False'
**Alert to be generated.
```

✅ Test 3: Accepted Password (Night Time)

Input:

Accepted password for ubuntu

Result:

- Rule ID: 17101
- Level: **5**
- Consistent behavior across login methods

```
Jan 20 23:30:00 clientmachine sshd[3333]: Accepted password for ubuntu from 192.168.50.100 port 54321 ssh2

**Phase 1: Completed pre-decoding.
  full event: 'Jan 20 23:30:00 clientmachine sshd[3333]: Accepted password for ubuntu from 192.168.50.100 port 54321 ssh2'
  timestamp: 'Jan 20 23:30:00'
  hostname: 'clientmachine'
  program_name: 'sshd'

**Phase 2: Completed decoding.
  name: 'sshd'
  parent: 'sshd'
  dstuser: 'ubuntu'
  srcip: '192.168.50.100'
  srcport: '54321'

**Phase 3: Completed filtering (rules).
  id: '17101'
  level: '5'
  description: '          Successful login during non-business hours (severity reduced)'
  ,
  groups: '['linux', 'ssh', 'override_demo']'
  firetimes: '2'
  mail: 'False'
**Alert to be generated.
```

✅ Test 4: Weekend Login

Input (Weekend Timestamp):

Accepted password for ubuntu

Result:

- Rule ID: 17102
- Level: **5**
- Weekend logic confirmed

```
Jan 18 11:30:00 clientmachine sshd[4444]: Accepted password for ubuntu from 192.168.50.100 port 55555 ssh2

**Phase 1: Completed pre-decoding.
full event: 'Jan 18 11:30:00 clientmachine sshd[4444]: Accepted password for ubuntu from 192.168.50.100 port 55555 ssh2'
timestamp: 'Jan 18 11:30:00'
hostname: 'clientmachine'
program_name: 'sshd'

**Phase 2: Completed decoding.
name: 'sshd'
parent: 'sshd'
dstuser: 'ubuntu'
srcip: '192.168.50.100'
srcport: '55555'

**Phase 3: Completed filtering (rules).
id: '17101'
level: '5'
description: 'Successful login during non-business hours (severity reduced)'
groups: '["linux", "ssh", "override_demo"]'
firedtimes: '2'
mail: 'False'

**Alert to be generated.
```

Result After Tuning

In Wazuh

- Same detections
- Lower severity
- Alerts still searchable and visible

In TheHive

- No alert flood
- Analysts focus on real incidents

```
> Jan 20, 2026 @ 10:17:58.097 | predecoder.hostname: clientmachine | predecoder.program_name: sshd | predecoder.timestamp: Jan 20 04:47:56 | input.type: log | agent.ip: 192.168.50.100 | agent.name: clientmachine | agent.id: 002 | manager.name: ip
data.srcip: 10.0.1.197 | data.dstuser: ubuntu | data.srcport: 37907 | rule.firedtimes: 1 | rule.mail: false | rule.level: 5 | rule.description: Successful login during non-business hours (severity reduced) | rule.groups: linux, ssh, override_demo | rule.id: 17101 | location: journal | decoder.parent: sshd | decoder.name: sshd | id: 1768804478.151510 | full_log: Jan 20 04:47:56 | sshd[2373]: Accepted publickey for ubuntu from 10.0.1.197 port 37907 ssh2: ED25519 SHA256:uJ5nTvQ8Bj24H05SFydaSeoAb5/6RV1BAK47F30I | timestamp: Jan 20, 2026 @ 10:17:58.097 | _index: wazuh-alerts-4.x-2026.01.20

> Jan 20, 2026 @ 10:17:58.097 | predecoder.hostname: clientmachine | predecoder.program_name: sshd | predecoder.timestamp: Jan 20 04:47:56 | input.type: log | agent.ip: 192.168.50.100 | agent.name: clientmachine | agent.id: 002 | manager.name: ip
data.srcuser: ubuntu | data.uid: 0 | data.dstuser: ubuntu | rule.firedtimes: 2 | rule.mail: false | rule.level: 5 | rule.description: Successful login during non-business hours (severity reduced) | rule.groups: linux, ssh, override_demo | rule.id: 17101 | location: journal | decoder.parent: pam | decoder.name: pam | id: 1768804478.151929 | full_log: Jan 20 04:47:56 | sshd[2373]: pam_unix(sshd:session): session opened for user ubuntu(uid=1000) by ubuntu(uid=0) | timestamp: Jan 20, 2026 @ 10:17:58.097 | _index: wazuh-alerts-4.x-2026.01.20
```

What If a Real Attacker Logs In?

This tuning **does NOT** reduce security.

A real attacker will generate **additional signals**, such as:

- Privileged command execution (sudo)
- Account modifications
- SSH key changes
- Persistence mechanisms

These actions:

- Trigger **separate high-severity rules**
- Still appear in **TheHive**
- Can be correlated with the login event

 The login alert becomes **context**, not noise.

SOC Correlation Strategy

SCENARIO	OUTCOME
LOGIN ONLY	Informational
LOGIN + SUDO	High severity
LOGIN + PERSISTENCE	Critical
LOGIN + BRUTE-FORCE HISTORY	Incident

This approach matches **real SOC operations**.

Conclusion

This project demonstrates how **small, safe rule changes** can have a **large operational impact**.

Achievements

- Reduced alert fatigue
- Preserved visibility
- Improved SOC efficiency
- Maintained detection integrity

Key Takeaway

Effective SOC tuning is about prioritization, not suppression.
